

人工智能基础 一轮复习

Notes:以下内容, 旨在尽可能扩充 `train_data` 从而控制 `train_loss`, 不排除: 发生了过拟合导致泛化能力下降。不考虑引入任何的正则化方法, 请读者自行评估, 审时度势。

CH1 人工智能基础绪论

CH2 机器学习与前馈神经网络 *

CH3 卷积神经网络 *

CH4 循环神经网络 *

CH5 网络优化与正则化

CH6 注意力机制

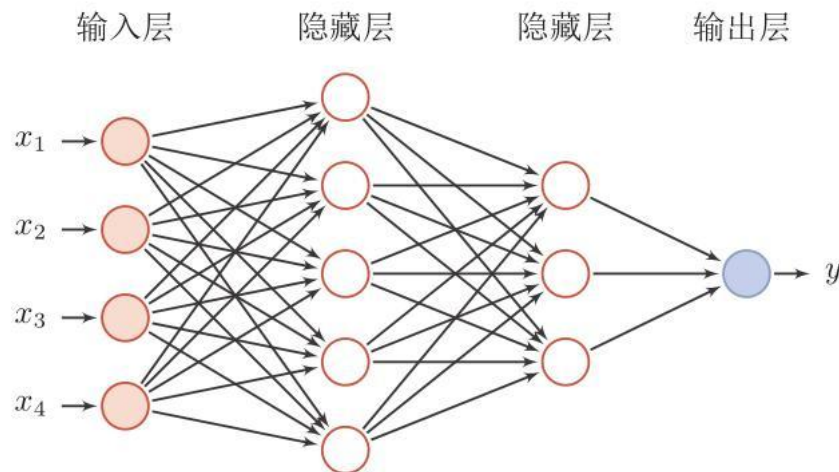
CH7 深度生成模型

CH8 深度强化学习

CH2 机器学习与前馈神经网络

1. 机器学习——以 线性回归为例：
2. 前馈神经网络

基本概念：信息仅在一个方向上流动，从输入层通过隐藏层最终到达输出层，信息传递过程中，**没有反馈循环**。



Ex 2.2.1 神经网络中的基本计算单元(基本组成单元)是？（ ）

- A. 神经元
- B. 激活函数
- C. 权重
- D. 损失函数

Ex 2.2.2 深度学习之所以“深”，是因为它通常包含？（ ）

- A. 更多的输入层
- B. 更多的隐藏层
- C. 更大的数据集
- D. 更长的训练时间

Ex 2.2.3 在神经网络中，哪个层用于将输入数据映射到隐藏层特征空间？（ ）

- A. 输入层
- B. 隐藏层
- C. 输出层
- D. 池化层

Ex 2.2.4 在神经网络中，_____是连接两个神经元的重要部分。

权重；

Ex 2.2.5 神经网络由许多神经元（Neuron）组成， 请问下列关于神经元的描述中，哪一项是正确的？（ ）

- A.每个神经元可以有一个输入和一个输出 B.每个神经元可以有多个输入和一个输出
C.每个神经元可以有一个输入和多个输出 D.每个神经元可以有多个输入和多个输出

FNN 信息处理流程：前向传播→损失计算→反向传播→优化算法；

对第 $l-1$ 层**前向传播**，生成预测值 $\hat{y} = x^l$ ： $x^l = \text{ReLU}(\mathbf{k}x^{l-1} + \mathbf{b})$ ；其中，类似于 $\text{ReLU}(\cdot)$ 的**激活函数**，是神经网络中非常重要的组成部分，它向网络**引入非线性**特性，使网络能够学习复杂的函数；

损失函数：用于衡量网络**预测**与**实际**目标之间的差异，常见的损失函数包括**均方误差 (MSE)**、**交叉熵损失**等；~~在某些题目里边，损失计算被归类在反向传播的算法里边，审时度势；~~

反向传播：通过链式法则，从网络的输出层开始，逐层向后（反向）计算损失函数对于**每一个权重和偏置的梯度**。

优化算法：通过上述梯度，依据参数更新公式 $\omega = \omega - lr \times \frac{\partial L}{\partial \omega}$ ，其核心目的是利用梯度更新模型参数，从而最小化损失。具体的参见 CH05。

Ex 2.3.1 前向传播主要用于？（ ）

- A. 计算损失 B. 更新权重
C. 生成预测 D. 检查梯度

Ex 2.3.2 在反向传播算法中，梯度是从哪里开始反向传播的？（ ）

- A. 输入层 B. 输出层
C. 隐藏层 D. 权重层

Ex 2.3.3 在神经网络中，以下哪项**不是**反向传播算法的步骤？（ ）

- A、前向传播 B、随机选择权重更新方向
C、计算损失 D、反向传播梯度

C.选项有歧义，考到的时候审时度势一点；我们的 PPT 里边，损失计算单独的、独立出来的一个步骤；

Ex 2.3.4 在深度学习模型中，以下哪项**不是**激活函数的作用？（ ）

- A、引入非线性 B、增加网络的深度

C. 确定损失函数 D. 帮助模型学习复杂的模式

Ex 2.3.5 哪种激活函数常用于二分类问题的输出层？（ ）

<u>A. Sigmoid</u>	B. ReLU	C. Tanh	D. Softmax
任务类型	输出层激活函数	输出意义	损失函数常用
回归	ReLU	任意实数值	均方误差
二分类	Sigmoid	一个概率值	二元交叉熵
多分类单标签	Softmax	一个概率分布	分类交叉熵
多标签分类	Sigmoid（每节点）	多个独立的概率值	二元交叉熵

Ex 2.3.6 深度学习中，ReLU 激活函数的优点不包括以下哪一项？（ ）

A. 计算效率高 B. 可以有效缓解梯度消失问题

C. 对输入数据有范围限制 D. 使得网络具有稀疏性

A. $ReLU(x) = \max(0, x)$ 只涉及比较和取最大值，没有指数、除法等复杂运算，计算速度比 Sigmoid / Tanh 快很多。

B. 在正区间梯度恒为 1，不会因为层数加深导致梯度指数级缩小，因此缓解了梯度消失（尤其在正输入时）。

C. $ReLU(x)$ 输入可以是任意实数，并不对输入数据做范围限制。关注到，Sigmoid / Tanh 会把输入压缩到固定区间。

D. 负半区输出为 0，可以让网络中的部分神经元输出为 0，增加稀疏性，减少参数依赖，可能提高泛化能力。

Ex 2.3.7 ReLU 激活函数相比 Sigmoid 的优点不包括？（ ）

A. 缓解梯度消失问题 B. 计算简单

C. 输出范围有限 D. 加快收敛速度

Ex 2.3.8 Softmax 函数通常用于？（ ）

A. 二分类问题 B. 多分类问题的输出层

C. 回归问题 D. 隐藏层激活

Ex 2.3.9 交叉熵损失函数常用于哪种类型的任务？（ ）

A. 回归 B. 分类

C. 聚类

D. 降维

在分类问题中，网络的输出通常经过 **Softmax**（多分类）或 **Sigmoid**（二分类）转化为概率分布，然后与真实标签的 **one-hot** 分布计算交叉熵。

回归任务常用均方误差（MSE）等损失函数。

聚类和降维属于无监督学习，一般不用交叉熵损失。

Ex 2.3.10 **图灵完备**：所有的图灵机都可以被一个由使用_____激活函数的神经元构成的_____来进行模拟。

Sigmoid ； 全连接循环网络；

CH3 卷积神经网络

全连接前馈神经网络 FNN(FC)缺点:

将图像展开为 Tensor 会丢失空间信息;

大量的参数也很快会导致网络过拟合;

参数过多效率低下, 训练困难;

卷积神经网络综述:

1. 采用**局部连接**和**权重共享**的方式, 减少权值的数量和减低模型复杂度;
局部连接: 卷积层的节点仅仅和其前一层的部分节点相连接, 只用来学习局部特征。
权重共享: 在同一个卷积层中, 同一个卷积核会滑过输入数据的每一个位置, 并且它在不同位置使用的权重参数是相同的。
权重共享 减少参数数量、提高计算效率、捕捉全局特征;
2. 在原来多层神经网络的基础上, 加入了**特征学习**;
3. 空间或时间上的**下采样**;

感受野: 是卷积神经网络每一层输出的特征图上每个像素点**映射回**输入图像上的区域大小。

$$RF_i = (RF_{i+1} - 1) \times stride_i + Kernel$$

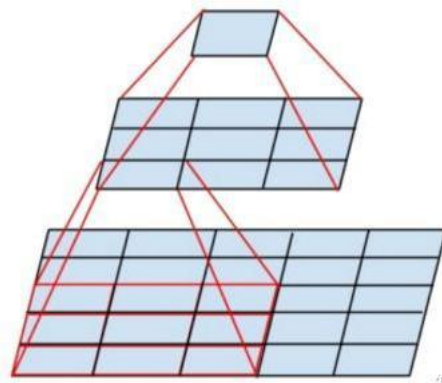
你看到我, 我身后是整个世界, 你的感受野是我, 我的感受野是整个世界;

感受野二级结论: 最后一层感受野的大小是**最后一层卷积核**的大小;

如何增加输出单元的感受野?

增加卷积核的大小; 增加模型层数来实现; 在卷积之前进行汇聚操作;

耍赖方法: 通过给卷积核插入“空洞”来变相地增加感受野的大小。



Ex 3.1.1

任务类型	输出层激活函数	损失函数	公式&说明
多分类 (图像分类)	Softmax (输出概率分布)	交叉熵损失	$L = - \sum_{c=1}^M y_{o,c} \log(p_{o,c})$ M: 类别数, y: 真实标签

		(one-hot), p : 预测概率
二分类 (真/假)	Sigmoid (输出 0~1 概率值)	二元交叉熵 $L = -[y \log(p) + (1-y) \log(1-p)]$ y : 真实标签(0 或 1), p : 预测为 1 的概率
回归 (预测坐标)	ReLU (直接输出实数值)	均方误差 $L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$ y : 真实值, $\hat{y}$: 预测值
多标签分类 (一张图多个 标签)	多个 Sigmoid (每个类别独立判断)	多标签交叉熵 将二元交叉熵损失对每个标签求和或平均

Ex 3.1.2 以下哪项**不是**卷积神经网络 (CNN) 的特点? ()

- A、能够学习空间层级的特征 B、可以自动学习到卷积核
C、需要大量参数 D、适用于处理图像数据

Ex 3.1.3 下列哪种神经网络结构最适合处理图像数据? ()

- A. 循环神经网络 (RNN) B. 卷积神经网络 (CNN)
C. 多层感知机 (MLP) D. 自编码器 (Autoencoder)

Ex 3.1.4 下列哪个神经网络结构会发生权重共享? ()

- A、卷积神经网络 B、循环神经网络
C、全连接神经网络 D、选项 A 和 B

Ex 3.1.5 (多选)下列关于卷积的理解**正确**的是 ()。

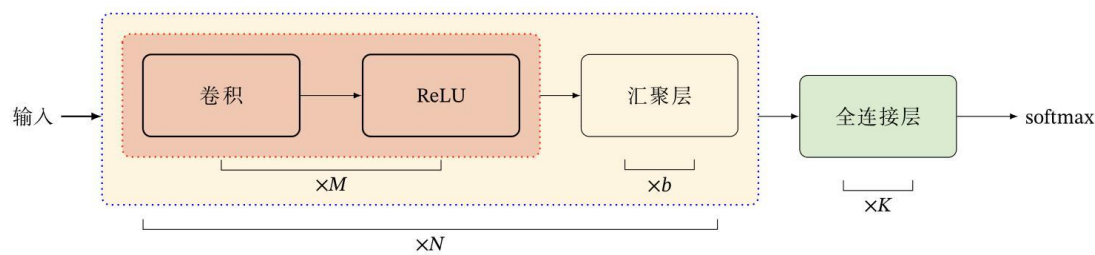
- A、卷积能抽取特征;
B、卷积过程是在图像每个位置进行线性变换映射成新值的过程, 将卷积核看成权重;
C、卷积 (相关) 运算是在计算每个位置与该模式的相似程度, 或者说每个位置具有该模式的分量有多少, 当前位置与该模式越像, 响应越强;
D、多层卷积是在进行逐层映射, 整体构成一个复杂函数, 训练过程是在学习每个局部映射所需的权重, 训练过程可以看成是函数拟合的过程;
E、卷积神经网络每层的卷积核权重是由数据驱动学习得来, 不是人工设

计的，人工只能胜任简单卷积核的设计，像边缘；

Answer: @All;

- A. 卷积通过不同的卷积核，对输入进行滤波，从而提取不同层次的特征；
- B. 卷积本质是局部区域的线性加权求和，卷积核的值就是权重；
- C. 相关运算就是模式匹配，输出值越大表示局部图像与卷积核模式越相似；其中，卷积在深度学习里不翻转核。对于神经网络来说，完全可以通过调整偏置项 (Bias) 来补偿这种偏移，因此这个区别在实践中被完全忽略了。
- D. CNN 通过多层卷积非线性激活，构成复合函数，用数据学习权重，本质是函数拟合；
- E. 传统图像处理中人工设计卷积核，但在 CNN 中卷积核权重通过反向传播从数据中学习，可以自动得到更复杂的特征检测器；

卷积神经网络模型架构：你好~ 汉堡王！



输入层 Input 卷积层 Conv 池化层 Pooling 全连接层 FC 输出 Output;

其中，输入层将原始图像数据转换为适合网络学习的格式和范围的过程，包括尺寸调整、归一化、去均值等步骤；

卷积层 Conv 对某个局部实现加权求和，对应局部感知，提取了局部特征；特征输出维度满足公式： $output = \lfloor (input + 2 \times padding - kernel) / stride \rfloor + 1$ 。什么窄卷积、宽卷积、等宽卷积，代入进去算就得了。

如果 padding 设置地不太好的话，Conv 其实也能实现下采样。诸如全卷积神经网络 FCN，就是不带 Pooling 直接 Conv，通过 Conv 实现下采样；

池化层 Pooling 执行了下采样，从而减少了每个通道上的神经元数量，即减少了特征映射的神经元个数；

Ex 3.2.1 分析 input.shape; output.shape; conv_layer.shape;


```

import torch

in_channels = 5 #输入通道数量
out_channels = 10 #输出通道数量
width = 100     #每个输入通道上的卷积尺寸的宽
height = 100    #每个输入通道上的卷积尺寸的高
kernel_size = 3 #每个输入通道上的卷积尺寸
batch_size = 1  #批数量

input = torch.randn(batch_size, in_channels, width, height)
conv_layer = torch.nn.Conv2d(in_channels, out_channels, kernel_size=kernel_size)

out_put = conv_layer(input)

print(input.shape)
print(out_put.shape)
print(conv_layer.weight.shape)

```

输入没啥说的， $X \in \mathbb{R}^{B \times H \times W \times C}$ ，其中 $B=1$ ； $H=100$ ； $W=100$ ； $C=5$ ；输出张量形状是：(batch_size, in_channels, width, height)，输出 torch.Size([1,5,100,100])；

卷积，定睛一看，kernel_size = 3；换句人话就是 $H=3$ ； $W=3$ ； $C=5$ ；卷积层输出张量形状是：(out_channels, in_channels, kH , kW)，输出 torch.Size([10,5,3,3])；

输出，公式做题， $\lfloor (100 + 2 \times 0 - 3) / 1 + 1 \rfloor = 98$ ，通道数变为 out_channels = 10，batch_size 不变，输出张量形状是：(batch_size, out_channels, width, height)，输出 torch.Size([1,10,98,98])；

Ex 3.2.2 卷积神经网络中的卷积层主要用于？（ ）

A. 提取特征 B. 减少参数数量 C. 增大感受野 D. 生成预测

Ex 3.2.3 在卷积神经网络中，以下哪个层用于降低特征图的维度？（ ）

A. 卷积层 B. 池化层 C. 全连接层 D. 批量归一化层

Ex 3.2.4 卷积神经网络（CNN）中的池化层（Pooling Layer）的主要作用是_____和_____。

实现下采样；维持图像的平移不变形；

Ex 3.2.5 输入图片大小为 200×200 ，依次经过一层卷积（kernel size 5×5 ，padding 1，stride 2），pooling（kernel size 3×3 ，padding 0，stride 1），又经过 1 层卷积（kernel size 3×3 ，padding 1，stride 1）之后，输出特征图大小为（ ）

$$output = \lfloor (input + 2 \times padding - kernel) / stride \rfloor + 1;$$

从运算的角度来讲，可以最后再做向下取整；

Conv1 $(200+2\times1-5)/2+1=99.5$ 向下取整 99；

Pooling $(99+2\times0-3)/1+1=97$ 向下取整 97；

Conv2 $(97+2\times1-3)/1+1=97$ 向下取整 97；

输出特征图大小为 97×97 ；

Ex 3.2.6 假设卷积神经网络某隐层的特征图大小是 $19\times19\times8$ ，其中 8 是通道数，使用大小为 3×3 的 12 个卷积核，步长为 2，没有 padding 对此隐层进行操作，得到的特征图大小是？（ ）

Conv $(19+2\times0-3)/2+1=9$ 向下取整 9；

输出特征图大小为 $X \in \mathbb{R}^{H\times W\times C}$ $H\times W\times C = 9\times9\times12$ ；

Ex 3.2.7 假设卷积神经网络某卷积层的输入和输出特征图大小分别为 $63\times63\times16$ 和 $33\times33\times64$ ，卷积核大小是 3×3 ，步长为 2，那么 Padding 值为多少？（ ）

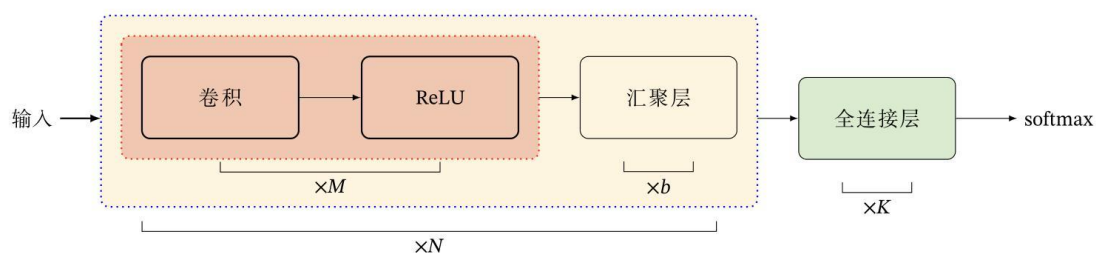
A. 0 B. 3 C. 2 D. 1

$$output = \lfloor (input + 2 \times padding - kernel) / stride \rfloor + 1;$$

代入数据 $33 = \lfloor (63 + 2 \times padding - 3) / 2 \rfloor + 1$ ，解得 padding=2；

Ex 3.2.8 有关一般卷积神经网络的组成，下面哪种说法是正确的？（ ）

- A. 卷积神经网络的层次结构依次是由输入层、卷积层、池化层、激活层和全连接层组成；
- B. 卷积神经网络的层次结构依次是由输入层、池化层、卷积层、激活层和全连接层组成；
- C. 卷积神经网络的层次结构依次是由输入层、卷积层、激活层、池化层和全连接层组成；
- D. 卷积神经网络的层次结构依次是由输入层、激活层、卷积层、池化层和全连接层组成；



Ex 3.2.9 有关卷积神经网络的说法哪个是正确的？（ ）

- A. 在卷积层后面使用池化操作，可以减少网络可以训练的参数量；
 - B. 1×1 的卷积没有改变特征图的大小，因此没有获得新的特征；
 - C. 不同卷积层或同一卷积层只能用一种大小的卷积核；
 - D. 类似 AlexNet 网络使用的分组卷积可以增加卷积层的参数量，降低网络训练速度；
-
- A. 池化（如最大池化）会减小特征图的尺寸（高和宽），从而减少下一层卷积或全连接层的参数数量；
 - B. 1×1 卷积虽然不改变高和宽，但可以改变通道数，并且通过非线性激活函数实现跨通道的信息整合与交互，从而得到新特征；
 - C. 同一层可以用不同尺寸的卷积核，不同层更可以用不同尺寸的卷积核；
 - D. 分组卷积的作用是减少参数量和计算量，从而加快训练速度；

CH4 循环神经网络

如何给网络增加记忆能力？

延时神经网络 TDNN，通过建立一个**额外的延时单元**，来存储网络的历史信息；

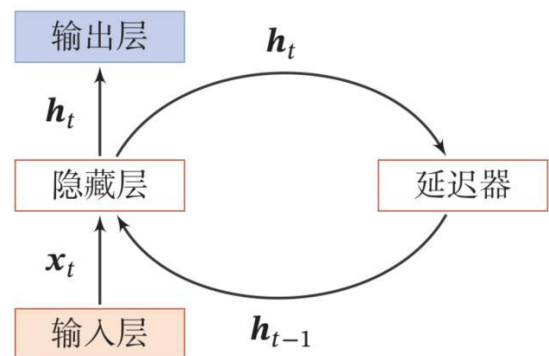
自回归模型 ARM 通过用**过去的的数据**来预测未来的数据；

非线性自回归模型 NARM，当前值不仅取决于自身过去的值，还取决于其他外部因素过去的值，并且这些关系可能是非常复杂的非线性关系；

循环神经网络通过使用**带自反馈的神经元**，能够处理任意长度的时序数据。

循环神经网络中节点或层的输出不仅传递给下一层，同时也作为输入反馈给自身（或同层的其他节点）。

循环连接指的就是从隐藏层输出端，经过延迟器，再返回到隐藏层输入端的整个闭环路径。



输入层：RNN 能够接受一个输入序列并将其传递到隐藏层。

隐藏层（循环层）：隐藏层之间存在循环连接，使得网络能够维护一个“记忆”状态。这一状态包含了过去的信息，这使得 RNN 能够理解序列中的上下文信息。

输出层：RNN 可以**有一个或多个输出**，例如在序列生成任务中，每个时间步都会有一个输出。

Ex4.1.1 假设输入序列是二维向量，偏置为 0，激活函数是线性 $f(x) = x$

初始隐藏状态 $h_0 = [0,0]$ ；其中

$$x_1 = [1,1], x_2 = [1,1], x_3 = [2,2]$$

$$W_{xh} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, W_{hh} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, W_{hy} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

展开来算就是了：

$$h_1 = W_{xh}x_1 + W_{hh}h_0 = [2,2]; \quad y_1 = W_{hy} h_1 = [4,4];$$

$$h_2 = W_{xh}x_2 + W_{hh}h_1 = [6,6]; \quad y_2 = W_{hy} h_2 = [12,12];$$

$$h_3 = W_{xh}x_3 + W_{hh}h_2 = [16,16]; \quad y_3 = W_{hy} h_3 = [32,32];$$

在机器学习领域的应用案例：

1. 从序列到类别 使用 LSTM 对 IMDB 进行电影影评分析，最终映射到 Negative 和 Positive 的两个类别；其中，嵌入层 编码器 分类器 输出层；

2. 同步的序列到序列模式；

这个过程像“贴标签”，输入一个序列，同时为序列中的每一个元素输出一个对应的标签。输出 y_t 只依赖于整个输入序列 X ，而不依赖于其他输出 y 。

输入输出等长，并行计算速度快，计算效率高；

BiLSTM-CRF：BiLSTM 捕捉上下文信息，CRF 层考虑标签之间的转移规则（比如“B-PER”后面不应该接“I-ORG”），是 NER 的经典模型。

BERT + 线性分类层：用预训练的 BERT 模型获取强大的上下文表征，然后接一个简单的分类层为每个 Token 打标签。这是目前非常主流的方法。

经典任务包括：分词、NER、词性标注、语音识别；

3. 异步的序列到序列模式；

像“做翻译/写摘要”，输入一个序列，输出一个全新的、长度自由的序列。输出 y_t 同时依赖于输入 X 和之前的所有输出 $(y_1, y_2, \dots, y_{t-1})$ ；

输入输出不等长，顺序生成，能捕捉输出序列内部的依赖关系；但处理速度慢，无法并行计算整个输出序列。

LSTM/GRU Seq2Seq（带注意力）：经典的编码器-解码器架构。编码器把输入编码成一个向量，解码器基于该向量和注意力机制，一步步生成输出。

Transformer：完全基于自注意力机制的模型，现在是异步任务（如机器翻译、文本生成）的绝对主流。它在训练时通过掩码实现自回归生成。

经典任务包括：机器翻译、文本摘要、对话生成、代码生成；

传统循环神经网络（RNN）会存在长程依赖问题。长程依赖问题是指——RNN 难以学习和记忆序列中相距较远的信息之间的关联，其根本原因是：随着时间步的推移，梯度在反向传播中会变得过小（梯度消失）或过大（梯度爆炸）。

长短期记忆神经网络(LSTM) 拥有门控结构，用来有选择地增加或移除细胞状态中的信息，门控机制包括：遗忘门，输入门和输出门；

1. 假设当前的时刻为 t ，将上一个隐藏状态 h_{t-1} 和当前输入 x_t 进行拼接，生成信息

$\hat{x}_t$ ；拼接后的 $\hat{x}_t$ 送入遗忘门，遗忘门决定从过去的细胞状态 C_{t-1} 中丢弃哪些信

息得到 f_t 。

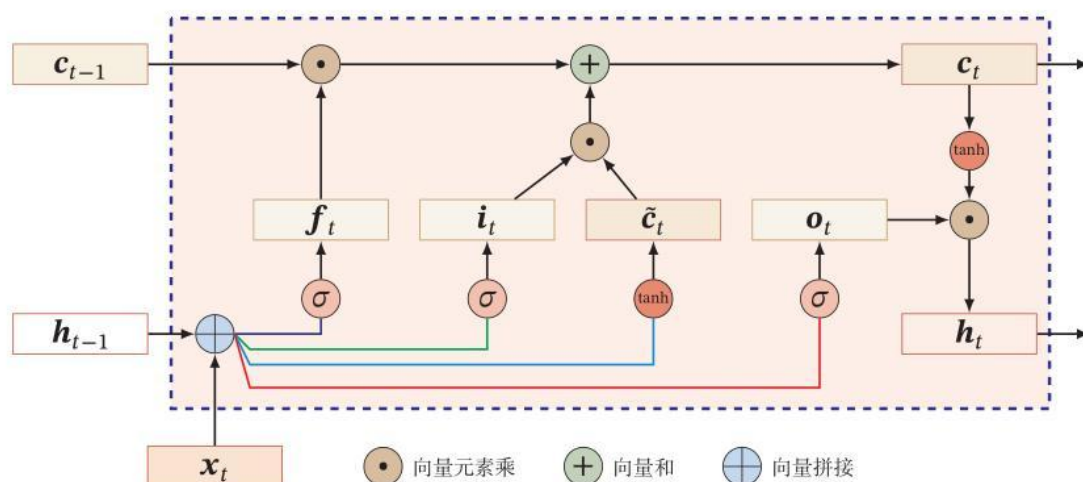
2. 上述拼接后的信息 $\hat{x}_t$ 同时送入输入门，将确定更新的部分 i_t 和候选值 $\hat{C}_t$ ；

3. 遗忘门输出&输入门输出，线性处理后存入当前细胞状态 C_t ，其过程可以表示

为： $C_t = f_t \cdot C_{t-1} + i_t \cdot \hat{C}_t$ ，实现了遗忘一部分旧的东西，加入一部分新的东西；

4. 基于新的 C_t ，通过输出门 o_t ，得到当前隐藏状态 h_t ，其过程可以表示为：

$h_t = o_t \cdot \tanh(C_t)$ ，并将其输出 h_t 传递给下一个时间步。

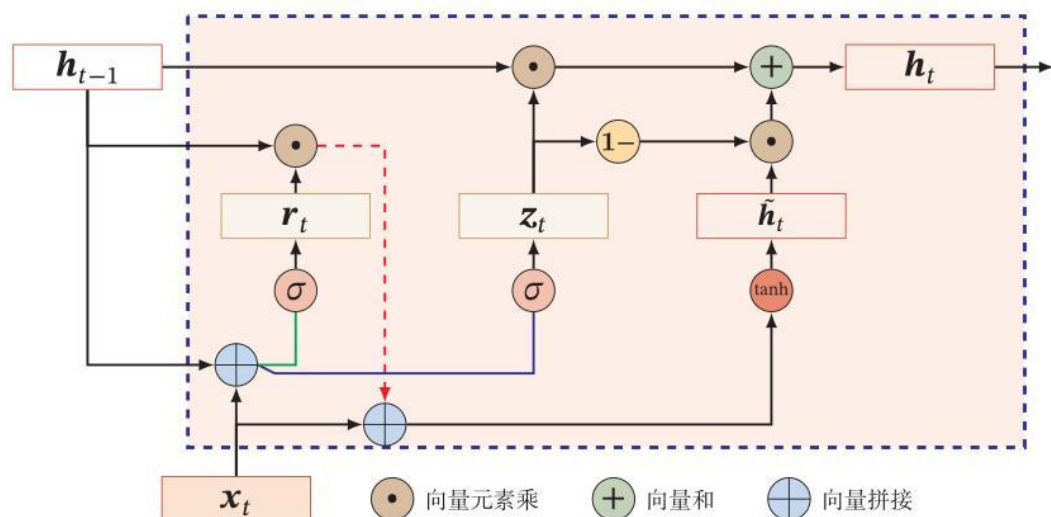


门控循环单元 (GRU)：是 LSTM 的一种简化版本，它通过“更新门”和“重置门”两个门控机制，在保持缓解长程依赖问题能力的同时，结构更简单，计算效率更高。

其中，重置门决定如何将新的输入信息与前面的记忆相结合，该过程可以表示为： $r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$ ；重置门在信号流中负责更新候选隐藏状态，也就是：

$$\tilde{h}_t = \tanh(W_c x_t + U(r_t \odot h_{t-1})) ;$$

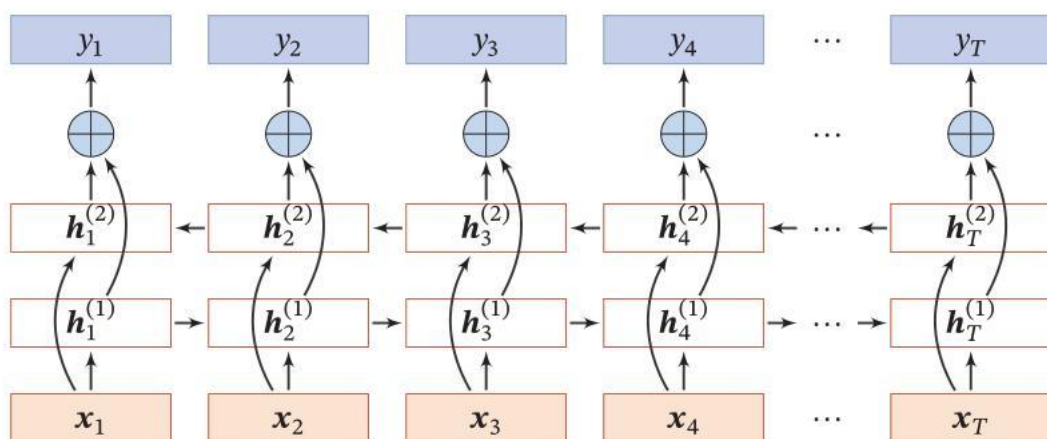
更新门用于控制前一时刻的状态信息被带入到当前状态中的程度，扮演了 LSTM 中“遗忘门”和“输入门”的角色，控制着有多少旧信息被保留，以及有多少新信息被加入。该过程可以表示为： $z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$ ；更新门在信号流中负责更新当前隐藏状态，也就是： $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$



双向循环神经网络：适用于任务不要求实时性且拥有完整的输入序列时；

通过同时使用一个前向 RNN 和一个后向 RNN，来捕捉序列中每一个元素的过去和未来的上下文信息，从而获得比单向 RNN 更丰富的表征。

核心优势：上下文更丰富，显著提升了对序列元素的理解能力，因为它能“看到”整个句子；性能更强，在拥有完整输入序列的任务上，其效果通常远优于单向 RNN；



Ex4.2.1 以下哪项不是循环神经网络（RNN）的变体？（ ）

- A、LSTM B、CNN
C、GRU D、Vanilla RNN

Ex4.2.2 以下哪项不是长短时记忆网络（LSTM）的结构特点？（ ）

- A、遗忘门 B、无法学习长期依赖
C、输入门 D、单元状态

Ex4.2.3 循环神经网络（RNN）在处理序列数据时，通常使用_____单元来解决梯度消失或梯度爆炸的问题。

长短期记忆（LSTM）或 门控循环单元（GRU）

Ex4.2.4 在处理序列数据时，哪种神经网络结构特别有效？（ ）

- A. CNN B. RNN
C. GAN D. Autoencoder

Ex4.2.5 以下哪个是递归神经网络的典型应用？

- A. 图像识别 B. 语音识别
C. 自然语言处理 D. 所有以上选项

Ex4.2.6 长短期记忆网络（LSTM）是一种特殊的循环神经网络（RNN），它通过引入_____和_____来克服传统 RNN 中的梯度消失和梯度爆炸问题，从而能够捕捉长期依赖关系。

门控机制；细胞状态；

Ex 4.2.7 **图灵完备**：所有的图灵机都可以被一个由使用_____激活函数的神经元构成的_____来进行模拟。

Sigmoid ； 全连接循环网络；

CH5 网络的优化与正则化

网络优化：让经验风险最小化；

难点 结构差异大；非凸优化问题；梯度消失（爆炸）问题；

解释 非凸优化问题：网络停留在一个鞍点上，以为到了终点，其实还有很长的下坡路可以走；一个找到平坦最小值的模型，通常比找到尖锐最小值的模型，在未见过的测试数据上表现更好，模型的泛化能力更强；

方法

1. 更有效的优化算法来提高优化方法的效率和稳定性

随机梯度下降 SGD：每次只随机抽取一个样本，用这个样本 计算的“随机”度来近似真实梯度，并立即更新参数。**内存效率高，收敛速度快，能够跳出局部最优；**
小批量随机梯度下降 MiniBatch：小批量样本数量发生改变 $batch \times k$ ，学习率也许需要进行对应的调整 $lr \times \sqrt{k}$ 。

学习率衰减策略：

固定衰减学习率：梯级衰减；线性衰减；

周期性学习率：三角循环学习率；带热重启的余弦退火；

自适应学习率：

Adagrad，实现了梯度平方累计和，解决稀疏数据优化问题；

RMSProp，梯度平方指数平均，解决 Adagrad 学习率衰减问题；

Adadelata，消除了学习率的超参数；

梯度优化方向：

Momentum 动量法，用之前积累动量来替代真正的梯度；

Nesterov 加速梯度，更加智能，可以减少震荡；

梯度截断方法，按模长执行截断；

自适应学习率+梯度优化方向：

Adam 算法，差不多是：动量法+RMSprop；

一阶矩估计计算动量，二阶矩估计计算自适应项；

2. 更好的参数初始化方法、数据预处理方法来提高优化效率

预训练初始化：

随机初始化：

Xavier 初始化：通常与 Tanh 或 Sigmoid 等 S 型激活函数配合使用效果更好，它的默认假设是——激活函数在零点附近是线性的。

He 初始化：专门为 ReLU 及其变体（Leaky ReLU）这类非线性激活函数设计。由于 ReLU 会将一半的输入置零，He 初始化通过调整方差来补偿信息损失，因此它在使用 ReLU 的网络中表现优于 Xavier 初始化。

固定值初始化：

数据的预处理:

批量归一化 BN: 对每个小批量的数据进行归一化, 使其均值为 0, 方差为 1; 对同一特征的不同样本进行归一化;

层归一化 LN: 在特征维度上归一化, 对单个样本的所有特征进行归一化;

3. 修改网络结构来得到更好的优化地形

ResNet 引入了残差连接; 用 $\text{ReLU}(\cdot)$ 作为激活函数; 分类时用交叉熵作为损失函数;

4. 使用更好的超参数优化方法

网格搜索; 随机搜索; 贝叶斯优化; 动态资源分配; 神经架构搜索;

Ex 5.1.1 以下哪项不是深度学习中常用的优化器? ()

A、SGD

B、牛顿法

C、Adam

D、均方误差

D 选项 绝对是错的, 均方误差是 Loss, 和优化器一点关系都没有;

B 选项 不太严谨, 深度学习里边不太用牛顿法, 计算量太大了;

Ex 5.1.2 在神经网络中, 以下哪项不是权重初始化的目的? ()

A、避免梯度消失

B、使所有权重相同

C、确保模型能够收敛

D、加速网络收敛

Ex 5.1.3 以下哪项不是批量归一化 (Batch Normalization) 的作用? ()

A、减少内部协变量偏移 B、允许使用更高的学习率

C、提高模型的泛化能力 D、减少模型的深度

Ex 5.1.4 哪种优化算法常用于深度学习的参数更新? ()

A. 梯度下降法

B. 牛顿法

C. 遗传算法

D. 模拟退火

余弦热启动退火和模拟退火, 他们不是一个东西~~

Ex 5.1.5 批量归一化通常应用于? ()

A. 输入层之前

B. 每个激活函数之前

C. 隐藏层之后, 激活函数之前

D. 输出层之后

隐藏层(FC / Conv)线性计算→批量归一化→激活函数 诸如 $\text{ReLU}(\cdot)$;

Ex5.1.6 在深度学习中，批量归一化（Batch Normalization）常用于解决什么？（ ）

- A. 过拟合。
- B. 欠拟合。
- C. 梯度消失。
- D. 梯度爆炸。

批量归一化通过稳定每层输入的分布，可以缓解内部协变量偏移，从而减轻梯度消失问题，允许使用更大的学习率，也能轻微起到正则化效果（减少过拟合），但它的主要设计目的是解决训练过程中梯度消失/不稳定问题。

网络正则化：降低模型复杂度；所有损害优化的方法都是正则化。

ℓ_1 正则化用来简化模型， ℓ_2 正则化防止过拟合；

Early Stop 机制：验证集（Validation Dataset）来测试每一次迭代的参数在验证集上是否最优；Validation Dataset 是从 Train Dataset 里边抽取出来的；

权重衰减：防止机器学习模型过拟合的技术，每次更新参数时，不仅根据梯度方向调整，还故意让权重值"衰减"一点点，防止它们变得过大；在标准的随机梯度下降 SGD 方法中，权重衰减正则化和 ℓ_2 正则化的效果相同

Dropout 机制：在训练过程中，随机"关闭"一部分神经元，让每次更新时网络结构都略有不同，从而防止过拟合。防止神经元共适应，体现模型的平均效应，增强模型的鲁棒性；

数据增强方法：图像数据的增强主要是通过算法对图像进行转变，引入噪声等方法来增加数据的多样性。

标签平滑：对原始的 one-hot 标签添加噪声，让模型不要过度自信，从而提高泛化能力。

Ex 5.2.1 深度学习中的 Dropout 技术的主要作用是：（ ）。

- A、防止过拟合
- B、增加模型复杂度
- C、提高计算效率
- D、减少参数数量

Ex 5.2.2 权重初始化技术如 He 初始化和 Xavier 初始化主要用于解决_____问题。

梯度消失 & 梯度爆炸问题；

Ex 5.2.3 防止神经网络过拟合的常用技术不包括？（ ）

- A. 增加数据量
- B. L1/L2 正则化
- C. 提前停止训练
- D. 增大网络规模

Ex 5.2.4 梯度爆炸问题通常与哪种因素有关？（ ）

- A. 学习率过大
- B. 数据量不足
- C. 网络层数过少
- D. 权重初始化不当

Ex 5.2.5 在深度学习中，哪种方法可以帮助提高模型的泛化能力？（ ）

- A. 增大网络规模
- B. 使用更多的训练数据
- C. 减小学习率
- D. 增加 Dropout 比例

Ex 5.2.6 在进行模型评估时，如果测试集上的性能远低于训练集，这通常意味着？（ ）

- A. 模型欠拟合
- B. 模型过拟合
- C. 数据集太小
- D. 损失函数选择不当

Ex 5.2.7 深度学习中，常用的优化算法包括梯度下降法、_____等。

题目给了 SGD；我们先写上 Adam；然后写拆开等价的 RMSProp、动量法；补上两个名字相近的 Adagrad；Adadelta

Ex 5.2.8 在深度学习中，哪种优化算法较为常用？

- A. 随机梯度下降
- B. 牛顿法
- C. 拟牛顿法
- D. 动量法

本题考验你的泛化能力，不论如何，B C 肯定不能选，然后 A 和 D 的话，我倾向于 A。A 是大名鼎鼎的 SGD，然后动量法的话，一般是在 SGD 基础上加入动量 Momentum，即 SGDM (SGD with Momentum)。诊断为题目不太好～

Ex 5.2.9 在深度学习中，下列哪种方法可以用于防止过拟合？（ ）

- A. 增加网络层数
- B. 减小学习率
- C. 使用更少的训练数据
- D. 引入 L2 正则化

CH6 注意力机制与外部记忆

既有的网络能力概述：全连接前馈网络→卷积神经网络→循环神经网络→图网络；网络能力逐项提升；

增加网络能力的另一种思路：引入注意力机制、外部记忆

注意力机制：

注意力打分函数：

键 输入向量 x ，可视为被匹配的标签；

值 输出向量 v ，是最终被提取的信息；

查询 查询向量 q ，表示当前的任务或问题焦点；

- 加性模型 $s(x, q) = v^T \tanh(Wx + Uq)$ ；

上述过程可以描述为：参数矩阵 W U 分别对输入键 x ，查询向量 q 进行映射，处理到维度为 d_h 的隐层空间；用 $\tanh$ 引入非线性；最后用维度 d_h 参数矩阵 v^T 与 $\tanh$ 的非线性输出做点积，得到一个标量分数。

参数量较多，达到 $d_h \times (d_x + d_q + 1)$ ；表达能力较强；

- 点积模型 $s(x, q) = x^T q$ ；

上述过程可以表示为：在输入键 x ，查询向量 q 同维度的基本条件下，通过点积操作来衡量它们在向量空间中的相似性；

计算简单高效，无参数量； x 与 q 要求维度相同；

- 缩放点积模型 $s(x, q) = \frac{x^T q}{\sqrt{D}}$ ；

当维度很高时，点积模型的数量级会很大，导致 softmax 激活之后的梯度很小，出现梯度消失问题；因此 Transformer 里引入了缩放因子 $\frac{1}{\sqrt{D}}$ ，将方差控制在 1 左右，有利于训练稳定。

计算简单高效，无参数量；不能学习复杂的匹配函数，灵活性低。

- 双加性模型 $s(x, q) = x^T W q$ ；

上述过程可以描述为：一个可学习的参数矩阵 W 同时对输入键 x 和查询向量 q 进行双线性交互映射，将二者直接通过矩阵 W 进行组合；该操作实质上是将 x 和 q 投影

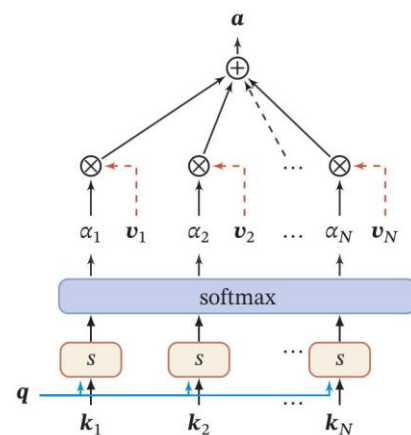
到一个共同的评分空间，无需引入非线性激活函数；最后通过 $x^T W q$ 的双线性形式直接计算出一个标量分数，表示二者之间的相关性。

参数量大；可以学习复杂的交互关系，灵活性高；适用于需要精细交互的任务；

软性注意力机制(Soft Attention)：假设输入向量可以表示为 $X=[x_1, x_2, \dots, x_n]$ ，那么软性注意力机制可以表示为： $att(X, q) = \sum_{n=1}^N \alpha_n x_n$ ；其中， α 是注意力分布。注意力分布根据打分函数在每一个维度上做激活，映射到每一维的概率分布，可以表示为： $\alpha_n = \text{soft max}(s(x_n, q))$ 。一句话的总结就是：注意力输出是对所有输入的加权和，权重本身通过打分函数给定出来。

硬性注意力机制(Hard Attention)：只选择一个输入，完全忽略其他；打分函数采用缩放点积模型；

键值对注意力机制(Key-value Pair Attention)：输入向量可以表示为 $(K, V) = [(k_1, v_1), (k_2, v_2), \dots, (k_n, v_n)]$ ，键值对注意力通过下式计算： $att((K, V), q) = \sum_{n=1}^N \alpha_n v_n$ ；其中， α 是注意力分布，满足 $\alpha_n = \text{soft max}(s(k_n, q))$ ，注意力



多头注意力机制(multi-head attention)：利用多个查询 $Q=[q_1, q_2, \dots, q_M]$ ，来并行地从输入信息中选取多组信息，每个注意力关注输入信息的不同部分。该过程可以表示为： $att((K, V), Q) = att((K, V), q_1) \oplus \dots \oplus att((K, V), q_n)$

结构化注意力：考虑注意力权重之间的结构关系，为注意力分布施加符合认知或数据内在规律的约束或先验结构。

自注意力机制：核心目标是是一种让序列中的每个元素都能够与序列中所有其他元素进行交互的机制。其核心目的，是通过序列自身来计算每个位置的加权表示，从而捕捉序列内部的依赖关系。

自注意力机制的 QKV 计算步骤可以表示为：输入 $\rightarrow$ QKV 投影 $\rightarrow$ 相似度计算 $\rightarrow$ softmax $\rightarrow$ 对 V 加权求和，即 $h_n = att((K, V), q_n)$ 。Transformer 等采用的自注意力机

制使用缩放点积作为打分函数，其最终输出是： $H = V \text{soft max}(\frac{K^T Q}{\sqrt{D_k}})$ ；当然，也可通过

多个注意力头从不同角度捕捉信息，形成多头自注意力机制(multi-head self attention)。

指针网络：它将**注意力分布**直接作为软性的指针，用于指示输入序列中相关元素的位置，而不是从固定词汇表中生成新的词汇。这使得模型能够学会“指向”或“引用”输入序列中的原始元素，特别适用于输出元素完全来源于输入序列的任务。

注意力机制应用场景：文本分类、自然语言推理、神经机器翻译、图像描述生成、阅读理解；

指针网络应用场景：排序、提取、组合优化等输出来自输入的任务；

Ex 6.1.1 简答题：比较加性注意力和点积注意力的优缺点；

Ex 6.1.2 简答题：给定 d_x, d_q, d_h ，算加性模型的参数量；

Ex 6.1.3 简答题：为什么加性模型要 $\tanh$ 和 v ？

Ex 6.1.4 简答题：为什么点积注意力需要缩放？加性注意力需要吗？

CH7 深度生成模型

深度生成模型：是利用神经网络构建生成模型；

主要分类：变分编码器 VAE 生成对抗网络 GAN

显式密度模型：明确地构建出数据样本的概率密度函数 $p(x|\theta)$ ，并通过最大似然估计 (MLE) 来学习模型参数 θ 。

典型模型：高斯混合模型 (GMM)、概率图模型 (如贝叶斯网络)；

隐式密度模型：不显式地定义数据分布的密度函数 $p(x|\theta)$ ，而是通过一个生成过程来直接生成符合数据分布的样本。

典型模型：变分自编码、生成对抗网络；

概率生成模型：一种包含隐变量 z 的概率图模型，将数据的生成过程建模为一个概率抽样过程。

EM 算法：引入一个关于隐变量的分布 $q(z)$ ，用来近似真实的后验分布 $p(z|x,\theta)$ ；

问题背景：我们有一个概率模型，其中既包含观测变量 x ，也包含隐变量 z 。我们的既有方法，是通过最大似然估计来求解模型参数 θ ，即最大化 $\log p(x|\theta)$ 。现在的困难在于：这个边际似然 $\log p(x|\theta)$ ，往往很难直接计算或直接实现最大化，因为它需要对隐变量 z 进行积分或求和：

解决方法：
$$\log p(x|\theta) = \underbrace{E_{q(z)}[\log p(x, z|\theta) - \log q(z)]}_{ELBO(q, \theta)} + \underbrace{D_{KL}(q(z) \| p(z|x, \theta))}_{KL \text{散度}}；$$
其中，

$\log p(x|\theta)$ 是我们真正想要最大化的边际似然；KL 散度衡量我们引入的分布 $q(z)$ 与真实后验分布 $p(z|x, \theta)$ 之间的差异。 $ELBO(q, \theta)$ 是证据下界，核心理由是因为：KL 散度 ≥ 0 恒成立， $\log p(x|\theta)$ 的最小情况即为 $ELBO(q, \theta)$ 。

算法流程：

E 步 (Expectation Step) 期望步让下界 ELBO 尽可能紧，即让 KL 散度尽可能小。本质上，在计算：给定当前参数 θ 和观测数据 x 的条件下，隐变量 z 的后验概率分布。

通过固定参数 θ ，令： $q(z) = p(z|x, \theta)$ ，使得 KL 散度 = 0，此时 $ELBO = \log p(x|\theta)$

M 步 (Maximization Step) 最大化步在提升下界 ELBO，从而提升边际似然。固定分布 $q(z)$ ，寻找新参数 θ^{new} 来最大化 ELBO: $\theta^{\text{new}} = \arg \max_{\theta} E_{q(z)}(\log p(x, z | \theta))$

例如，在高斯混合模型 GMM 中，**观测变量** x 是我们看到的样本点；**隐变量** z 是每个样本点属于哪个高斯成分；**E 步** 计算每个样本点属于每个高斯成分的后验概率；**M 步** 用这些后验概率作为权重，重新估计每个高斯成分的均值、方差和混合权重。

Ex 7.1.1 解释显式密度模型和隐式密度模型的区别。

Ex 7.1.2 为什么 GAN 不能使用最大似然估计？

Ex 7.1.3 描述概率生成模型中隐变量 z 的作用。

变分自编码器 (Variational Autoencoder, VAE)：将神经网络与变分推断结合在一起，解决了 如何在一个复杂的概率生成模型中，高效地学习和推理？ 这一基本问题。

Encoder 引入变分分布 $q(z | x; \phi)$ ，由参数 ϕ 决定的分布 $q(z | x; \phi)$ 来近似真实的后验分布 $p(z | x; \theta)$

CH8 深度强化模型

1. 强化学习基础概念与术语

一个**智能体(Agent)**通过不断地在**环境**中尝试不同的**动作**，从状态转换到新的状态，并从环境获得**奖励反馈**，从而学习一个最优的策略，这个策略能最大化其从环境中获得的**长期累积奖励**。其中：

环境 (Environment) 是智能体所处的外部世界，它会对智能体的动作做出反应；

状态 (State) 是环境在某一时刻的具体情况或信息；

动作 (Action) 是指智能体在某个状态下可以执行的操作；

策略 (Policy)，是智能体的“行为准则”或“大脑”。它规定了智能体在任何一个给定的**状态**下，应该如何选择**动作**。**策略函数**不是固定的指令，而是一个**随机的概率分布**，表示给定状态做出动作的概率密度，可以表示为： $\pi(a|s) = P(\text{Action} = a | \text{State} = s)$

状态转移 (State Transition) 描述了环境如何响应智能体的动作而发生变化，它代表了环境自身的动态规则。状态转移是**随机的**，这种随机性**来自于环境，而不是智能体**。该过程可以表示为： $p(s'|s, a)$ ，其中， s' 是当前状态； s 前一时刻的状态； a 是从前一时刻 s 转移到当前状态 s' 采取的动作 a ；

奖励 (Reward) 是在智能体执行一个动作后，环境反馈给它的一个**标量信号**。奖励是智能体学习的**唯一指南**，用符号 R_t 表示。它的终极目标就是最大化长期获得的总奖励。

回报 (Return) 是指从当前时刻 t 开始，未来所获得的**所有奖励的总和**。智能体的目标不是最大化即时奖励 R_t ，而是最大化整个交互过程的总回报 U_t 。它关注的是**长期的、整体的成功**，可以表示为： $U_t = R_t + R_{t+1} + \dots + R_{t+n}$ 。在更多的情况下，我们考虑

Discounted Returns，该过程可以表示为： $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^n R_{t+n}$ 。

Ex8.2.1 解释 Policy 策略中，为何要设置成随机？

如果策略完全是确定性的，智能体就无法尝试其他可能更好的动作。通过**随机性**，智能体可以探索环境，发现新的、可能带来更高奖励的行为模式。同时，在部分可观测的环境中，状态本身可能具有不确定性，**随机性策略**可以提供更鲁棒的行为。

2. 强化学习核心思想与评估方法

随机性总结：在同一个状态下，智能体也可能根据其**随机性策略**选择不同的动作。不同的动作，会引发环境不同的反应，从而导致在未来获得不同的奖励序列。这也就代表着：动作随机→环境状态转移随机→整体的奖励序列随机。

状态值函数 $V^\pi(s)$ ：考虑马里奥游戏的例子：用我当前的策略，从“起点”这个状态开始玩，平均能得多少分？这个“平均能得多少分”，就是状态值函数 $V^\pi(s)$ 要回答的——从状态 s 出发，遵循策略 π 玩游戏，所能获得的折扣后总回报的“平均值”

（期望回报）是多少，具体表达式是：
$$V^\pi(s) = E_{\tau \sim p(\tau)} \left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \mid \tau_{s_0} = s \right]$$

其中， $\sum_{t=0}^{T-1} \gamma^t r_{t+1}$ 是指从游戏开始（ $t=0$ ）到游戏结束（ $t=T-1$ ）的所有打折后的奖励加起来； τ 代表一条**轨迹**，可以理解成玩一局游戏的完整记录，包括你看到了什么，你做了什么，得了多少分； $p(\tau)$ 是指在给定策略 π 下出现某条轨迹的概率，这个概率反映了上一节一再强调的随机性； $\tau_{s_0} = s$ 的含义是：轨迹的起始状态为 s ，可以理解成：我们固定了游戏的起点，比如每次都从这个“起点”开始玩。

状态-动作值函数 $Q^\pi(s, a)$ ：我们现在还想知道，在状态 s 下执行某个特定动作 a ，然后之后再遵循策略 π ，长期来看平均能得多少分呢？该过程表达式是：

$$Q^\pi(s, a) = E_{s' \sim p(s' | s, a)} [r(s, a, s') + \gamma V^\pi(s')]$$

贝尔曼方程 (Bellman Equation)：建立了当前**值函数**与后续**值函数**之间的关系，是几乎所有算法的理论基础。

3. 经典的强化学习方法

基于模型 (Model-based)：

策略迭代算法：通过交替执行两个关键步骤来实现目标，分别是**策略评估 (Policy Evaluation)** 和**策略改进 (Policy Improvement)**，直到策略不再变化为止。

值迭代算法：值迭代直接通过迭代更新**状态值函数**来找到最优策略，直接求解最优状态值函数 $V^*(s)$ 。

其中，基于模型的含义是：与智能体交互的环境是已知的，环境的状态转移和奖励函数是已知的。

其中，**策略迭代算法**对当前策略 π 进行评估，得到其状态值函数 $V^\pi(s)$ ；使用贝尔曼期望方程进行策略改进从而得到一个更好的策略。其中，更新策略为贪心策略，使用 $\pi(s) = \arg \max_a (Q(s, a))$ 使得在每个状态 s 都能选择到**能使 $Q(s, a)$ 最大的动作 a** 。

同时，**值迭代算法**中：使用贝尔曼最优方程进行迭代更新，可以理解成：对于每个状态，考虑所有可能的动作，选择能使**即时奖励 + 折扣后的未来奖励期望值最大**的那个动作对应的值；该过程可以表示为： $\max_a (E_{s' \sim p(s'|s, a)} [r(s, a, s') + \gamma V^\pi(s')]) \rightarrow V(s)$ 。重复上述流程，直到所有状态的值函数 $V(s)$ 不再发生显著变化（收敛）当值函数收敛后，再从最优值函数导出最优策略；

无模型 (Model-free)：智能体只能通过和环境进行交互，通过采样得到的数据进行学习；

蒙特卡罗方法：通过完整回合的**重复随机抽样**，来直接估计值函数。

时序差分学习：结合动态规划和蒙特卡罗，每一步利用当前获得的奖励+下一个状态的价值估计作为当前状态的回报。从而在线、增量式地更新值函数。

SARSA：在线策略学习 (on-policy)，通过实际与环境交互产生的连续状态-动作对 (s, a, r, s', a') ，来在线学习当前策略下的状态动作值 Q ，并同时使用这个 Q 值来优化策略。

Q-learning：离线策略学习 (off-policy)，通过假设在后续状态中始终采取最优动作，来直接评估并学习最优状态动作值 $Q^*(s', a')$ 来更新，并非每次都要求从改进后的贪心算法得到新的数据，是 DQN 的基础。

Q-learning 目标不稳定，样本之间有很强的数据相关性。

探索与利用： ϵ -贪心法 (ϵ -greedy)。

Q Learning 与 SARSA 的关键区别：在线策略学习 vs 离线策略学习：

特性	SARSA (同策略)	Q-learning (异策略)
学习目标	学习执行策略的值函数	学习最优策略的值函数
更新所用动作	a' 由行为策略 ϵ -greedy 产生	a' 由 $\max(Q)$ 产生
学到的策略	更安全、考虑探索的策略	更激进、理论的最优策略

当使用**神经网络**来近似**策略搜索**或价值函数时，我们就进入了**深度强化学习**的领域——用**强化学习**来定义问题和**优化目标**，用深度学习来解决状态表示、策略表示等问题。

4. 深度强化学习 - 基于值函数

经典 Q-learning 维护了一个巨大的表格 Q-table，为每一个状态 s 和每一个动作 a 存储一个值 $Q(s, a)$ 。但是，当状态空间或动作空间非常大或者是连续时，这个表格会变得无限大，无法存储和计算。这就是所谓的“维度灾难”问题；

我们用一个参数为 Q_θ 的模型来拟合或近似这个 Q 值，该模型的核心任务就是：学习一个函数，使得 $Q_\theta(s, a) \approx Q^\pi(s, a)$ 。这个模型就叫做 **深度 Q 网络 DQN**。

设置了目标网络冻结（freezing target networks），即在一个时间段内固定目标中的参数，来稳定学习目标；

构建了经验回放（experience replay），构建一个经验池来去除数据相关性。

经典 Q-learning 处理有限状态算法；DQN 用于解决连续状态问题；

5. 深度强化学习 - 基于策略函数

策略梯度：对策略函数 π_θ 建模，策略梯度方法直接优化策略参数 θ ，通过梯度上升最大化策略函数；这是一个方法论！

REINFORCE 算法：借助蒙特卡洛方法采样轨迹，用来估计状态动作值函数，可以得到无偏的梯度。

REINFORCE 算法问题：使用蒙特卡罗策略梯度，方差大，训练不稳定。

解决方法：引入基线 $b(s_t) = V_\phi(s_t)$ ，保持无偏性的同时，显著减小方差，加速收敛。

Actor_Critic 算法：是一种结合策略梯度和时序差分学习的强化学习方法，既能学习价值函数，又能学习策略梯度。

其中，演员（actor）是指策略函数，在评论员（critic）指导下，学习一个策略来得到尽量高的回报；

评论员（critic）是指值函数，对当前策略对应的值函数进行估计，即评估 actor 的好坏，帮助 actor 进行策略更新。

